

Energy Optimization in Software: A Comparative Analysis of Programming Languages and Code-Execution Strategies

Ismael Camargo-Henríquez 
 Universidad Tecnológica de Panamá
 Panamá, República de Panamá
 Email: ismael.camargo@utp.ac.pa

Anthony Martínez-Rojas 
 Universidad Tecnológica de Panamá
 Panamá, República de Panamá
 Email: anthony.martinez@utp.ac.pa

Gema Castillo-Sánchez 
 Universidad Tecnológica de Panamá
 Panamá, República de Panamá
 Email: gema.castillo@utp.ac.pa

Abstract — This study investigates the differences in energy consumption between various programming languages, comparing those executed natively with those run on a Virtual Machine. The results challenge existing assumptions about the energy efficiency of software development practices. The findings reveal a positive correlation between the code execution strategy and resource consumption. Additionally, the study identifies key factors influencing the energy efficiency of applications beyond the choice of programming language. This research offers valuable insights for developers aiming to optimize energy consumption in their projects and prompts further questions regarding the future of development platforms and sustainable programming practices.

Keywords — *programming languages, software sustainability, green software, energy efficiency, energy consumption.*

NOMENCLATURE

DRAM	Power consumption by the RAM.
RAM	Random access memory.
CPU	Power consumption by all processor cores during code execution.
PKG	Total power consumption by the processor, encompassing cores, integrated graphics, last-level caches, and the memory controller.

I. INTRODUCTION

Traditional programming education generally focuses on fundamental languages and methodologies designed to enhance technical skills, often overlooking the energy impact of software.

However, with the proliferation of mobile devices, cloud computing, and other technological advancements, it has become essential to understand the interplay between software and energy consumption [1]. Mobile devices are frequently constrained by battery life limitations [2], which can impact their processing performance. In contrast, data centers aim to reduce power consumption [3], even while employing advanced virtualization and infrastructure management techniques [4][5].

As a result, there is increasing recognition of the necessity to incorporate energy-efficient practices into software development [6][7][8][9].

In this context, the choice of programming language is crucial due to its direct impact on the development process and its significant effects on energy consumption.

Although the energy implications may not be immediately apparent, they are substantial. Software execution encompasses multiple stages, from the compilation of source code produced by the programming language to its execution on hardware. Each of these stages involves energy expenditure. Consequently, energy efficiency is influenced by factors such as the implementation and execution of the source code.

This study aims to provide a comprehensive overview of factors affecting the energy consumption of programming languages, with a specific focus on comparing two primary code execution strategies: Virtual Machine and Native Execution. The primary objective is to enhance the understanding of the energy implications of software design choices and to promote the development of more energy-efficient applications.

II. BASIC CONCEPTS

A. Understanding Software Power Consumption

To examine the energy consumption of programming languages, it is essential to understand the operation of a typical hardware and software stack, as depicted in Figure 1. It is important to recognize that electricity consumption is exclusively associated with hardware components.

For instance, the CPU performs calculations and modifies data through physical logic gates, while the RAM updates its memory cells, and the disk performs data read and write operations.

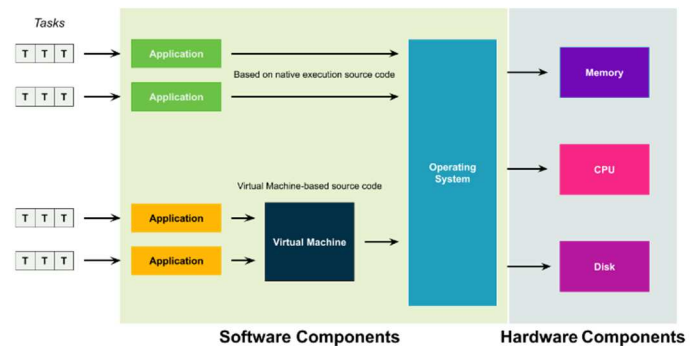


Fig 1. Energy model in a software-hardware stack.

Thus, software components, consisting of executable files stored on disk (including operating system), do not directly consume power. Instead, during execution, hardware loads these files from the disk into RAM and processes them using the CPU.

The power consumption is attributed to the hardware, driven by the execution of the software logic. Therefore, in this context, terms such as “*software power consumption*” or “*software energy usage*” refer to the energy consumed by hardware because of executing software. This may be influenced by the specific paradigms of software implementation discussed in this paper.

B. Execution of Source Code via Virtual Machine

A Virtual Machine is a software-based environment that emulates a complete computer system. It enables operating systems and applications to function in isolation, like running on separate physical machines. Virtual Machines are extensively utilized in software testing, server consolidation, and cloud computing environments [10].

When code is executed on a Virtual Machine, it is first translated into an intermediate representation known as bytecode, which is independent of the underlying hardware architecture. The Virtual Machine then interprets or compiles this bytecode in real-time, providing an abstraction layer between the software and the hardware [11] (see Fig. 1). This abstraction facilitates the high portability of the code, allowing it to run on various platforms and operating systems with minimal modifications [12].

C. Native Source Code Execution

In contrast to Virtual Machine execution, native execution involves compiling source code directly into machine-specific instructions tailored for a given processor and operating system. This compiled code runs directly on the platform without intermediary layers [13] (see Fig. 1).

Consequently, native execution typically provides higher performance, as it benefits from optimizations specific to the hardware and avoids the overhead associated with Virtual Machines.

D. Power Consumption and Performance

Software power consumption refers to the electrical power required for a program to execute its instructions on specific hardware. This consumption results from the internal operations of the processor, memory, and other system components activated by the software. Factors affecting this power usage include the choice of programming language, the efficiency of the compiler, the algorithms implemented, and the characteristics of the underlying hardware [14].

III. METHODOLOGY

This study is designed to address the following research question:

RQ₁: *How do the energy consumption and efficiency of source code execution vary between programming languages that rely on a Virtual Machine versus those that execute natively on hardware?*

A. Experimental Design

To evaluate the impact of execution strategy on energy consumption, this study selected three programming languages that use Virtual Machine-based execution—Python, C#, and Ruby—and three languages that execute natively—C, Rust, and Go.

For each language, a computational task was implemented that utilizes arbitrary precision arithmetic, based on the Rabinowitz and Wagon method [15], to sequentially generate digits of π (pi) using a single thread of execution.

$$\pi = \sum_{i=0}^{\infty} \frac{(i!)^2 2^{i+1}}{(2i+1)!} \quad (1)$$

In programming terms, the Rabinowitz and Wagon method, expressed in (1) was implemented as ALGORITHM 1.

```

1 Procedure ComputePiDigits(n):
2   Let A be an array initialized to [2, 2, ..., 2] with length (10n / 3) + 1
3   For i from 1 to n do:
4     // Step 1: Multiply each entry of A by 10
5     For each element in A do:
6       Multiply the element by 10
7   // Step 2: Put A into regular form
8   For i from length of A down to 1 do:
9     If i > 1:
10      // Reduce the i-th element modulo 2*i - 1
11      quotient, remainder = Divide (A[i - 1], 2*i - 1)
12      A[i - 1] = remainder
13      A[i - 2] = A[i - 2] + quotient * (i - 1)
14   // Step 3: Get the next digit
15   predigit = A[0] mod 10
16   A[0] = predigit
17   // Step 4: Adjust the digits
18   If predigit is not 9 and not 10 then:
19     Release all held predigits as true digits of n
20     Hold predigit q
21   Else If predigit is 9 then:
22     Add 9 to the queue of held predigits
23   Else If predigit is 10 then:
24     Set current predigit to 0 and hold it
25     For each held predigit:
26       increase it by 1 (* becomes 0)
27     Release all but the current held predigit as true digits of n
28 End

```

ALGORITHM 1. CALCULATION OF π (PI) USING THE RABINOWITZ AND WAGON APPROXIMATION.

B. Test Environment Specifications

The test environment consisted of a system to run an extensive range of applications and workloads. This system was equipped with 32 GiB of RAM to support memory-intensive tasks.

The central component of the system was an Intel Core i5-10210U processor, which operates at a base clock speed of 1.60 GHz and can boost up to 4.20 GHz when additional performance is required. The processor features four physical cores and supports simultaneous execution of up to eight threads. However, for this experiment, the algorithm was executed using a single thread.

The operating system used for this study was Linux Ubuntu 22.04.4 LTS, a version noted for its stability and long-term support. As a 64-bit distribution, it enabled full use of the system's processing and memory capabilities.

For estimating power consumption, the study employed RAPL (Running Average Power Limit) [16], a feature of Intel processors that provides accurate tracking of power consumption for the processor and other chip components.

Power consumption data were accessed through *Machine Specific Registers* (MSR) such as `MSR_PKG_Energy_Status` and `MSR_PP0_Energy_Status`.

These registers enable operating systems and applications to monitor and manage power consumption more effectively.

C. Test Execution and Implementation

For each selected programming language, ten executions of the Rabinowitz and Wagon algorithm [15] were conducted. The results from each run were recorded in comma-separated values (*.csv) files and aggregated in a spreadsheet for subsequent analysis.

The comparison method involved evaluating the performance of each language by executing the same task with the language's standard resources, excluding the use of external libraries. This approach allowed for a direct correlation between the results obtained and the energy consumption of the test environment's underlying architecture.

During the tests, four key variables were evaluated:

- **PKG:** This variable estimates the total power consumption of all processor cores, integrated graphics, and other non-core components such as last-level caches and the memory controller required by the executed code.
- **CPU:** This variable measures the power usage of all processor cores specifically during code execution.
- **DRAM:** This variable quantifies the power consumption of the random-access memory (RAM) connected to the integrated memory controller.

All measurements were recorded in *Joules* (J).

Additionally, the execution **Time** for each task was measured in milliseconds (ms).

For each programming language, the average execution was computed from the individual measurements obtained during each round. The results are summarized in TABLE 1.

TABLE 1. SUMMARY OF RESULTS FOR EACH PROGRAMMING LANGUAGE, CATEGORIZED BY EXECUTION STRATEGY.

	Language	PKG (J)	CPU (J)	DRAM (J)	Time (ms)
VM-Based Execution	PYTHON	25.9098755	14.7348328	2.87481079	1188.946
	C#	20.8189392	11.8302246	2.29424438	947.4443
	RUBY	199.817145	122.561194	26.5667786	8200.75
Native Execution	C	11.2728027	6.15424805	1.34835205	558.1377
	RUST	12.0921204	6.77314453	1.34835815	557.3961
	GO	19.4788635	11.245282	1.86621094	770.0474

This approach enabled a direct comparison of energy efficiency and performance between programming languages that utilize either a Virtual Machine-based execution strategy or a Native Execution strategy.

IV. RESULTS AND DISCUSSION

Figures 2 and 3 provide an illustrative representation of the data analyzed in TABLE 1.

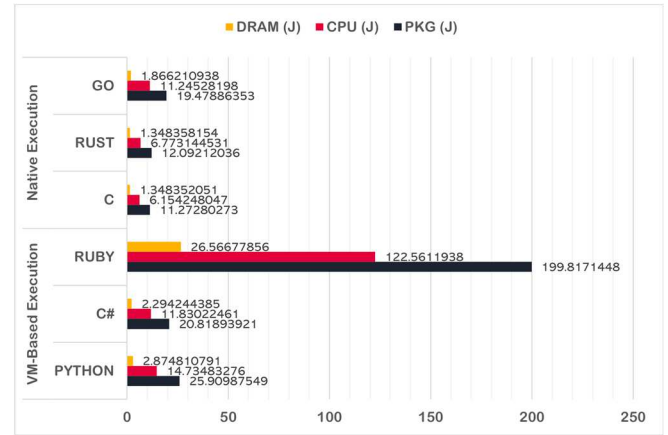


Fig 2. Comparison of the total energy, CPU, and DRAM consumption by programming languages based on their execution strategy.

Analyzing Fig. 2, reveals that Ruby exhibits higher total energy consumption, CPU usage, and DRAM consumption, along with significantly longer execution times (see Fig. 3). This suggests that Ruby may be less efficient both energy and performance for certain tasks.

Conversely, among Virtual Machine-based languages, C# demonstrates lower total power and CPU consumption compared to Python (see Fig. 2), along with shorter execution times, as shown in Fig. 3. While Python shows better efficiency compared to Ruby, C# emerges as the most efficient language in terms of both energy consumption and execution time among those tested.

Among the languages that use Native Execution (see Fig. 2), Go exhibits the highest total, CPU, and DRAM power consumption, along with the longest execution time as shown in Fig. 3. In contrast, C and Rust show similar power consumption levels, with C being slightly more efficient in total and CPU power consumption. Rust, however, has marginally better execution times compared to C (see Fig. 3), which may suggest a slight advantage in processing efficiency. Both C and Rust are more energy-efficient compared to Go.

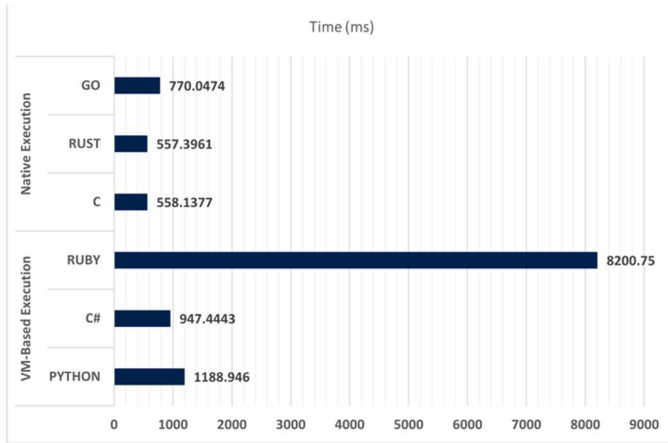


Fig 3. Comparison of runtime performance of the programming languages based on their execution strategy.

In summary, if prioritizing energy efficiency and execution time, Native Execution languages such as C and Rust are more efficient than those relying on Virtual Machines. This raises the question: *Is the Virtual Machine model suitable for sustainable code execution?*

Although Virtual Machines may introduce some overhead in power and memory consumption, they are not inherently unsustainable. Other factors, such as algorithm efficiency, code quality, and development practices, play a crucial role in enhancing overall energy efficiency.

A. Potential Biases

The study identifies several potential limitations and biases that may impact the validity and generalizability of the results. First, implementation dependence is a significant limitation.

For instance, performance and power consumption metrics can vary substantially depending on the specific Virtual Machine implementation.

Variability may exist between Virtual Machines for languages such as Python, and C#, as well as between different compilers. Consequently, different versions or configurations of these runtime environments could produce divergent results, which may not accurately represent the overall expected behavior.

Similarly, the influence of hardware in the testing context is also a significant factor in this study. The hardware architecture, with the operating system and other aspects of the physical environment, can significantly influence the outcomes.

Variations in hardware, such as differences in processing power or resource management (e.g., memory), can alter performance and power consumption, potentially affecting the applicability of the findings to different environments.

Finally, the nature of the Rabinowitz and Wagon algorithm, used as the testing framework, may introduce biases into the study. The specific characteristics of this algorithm, such as its dependency on processor performance, high memory usage, and extensive input/output (I/O) operations, may affect the results.

Consequently, variations in implementation with different usage and operational profiles could yield performance and energy consumption patterns that are not easily generalizable.

B. Limitations and Future Work

The results of this study are subject to limitations, including dependency on specific implementations and the influence of the testing environment. Future research should address these limitations by evaluating a wider range of programming languages and algorithms and investigating the effects of various hardware configurations. Additionally, examining the impact of code optimization techniques and performance analysis tools on application energy efficiency would be valuable.

CONCLUSIONS

This study compared the energy impact and performance of programming languages based on Virtual Machines versus Native Execution strategies.

The results indicate a significant influence of the execution strategy on both energy consumption and performance. Languages utilizing Native Execution, such as C and Rust, demonstrated greater efficiency in terms of total energy consumption and execution time compared to those relying on Virtual Machines.

Among the Virtual Machine-based languages, C# emerged as the most efficient. Conversely, Go exhibited the highest energy consumption among the Native Execution languages evaluated.

These findings highlight the importance of incorporating energy consumption considerations from the earliest stages of software development. Understanding the efficiency differences between various execution strategies and programming languages enables developers to make informed decisions that optimize application energy consumption and support sustainability goals.

Although the results indicate that Native Execution languages may offer better energy efficiency, Virtual Machines provide significant benefits in terms of portability and ease of development.

Additionally, factors such as algorithm efficiency and development practices are critical to application energy consumption. Therefore, selecting an execution strategy requires balancing efficiency, portability, and productivity.

REFERENCES

- [1] A. Katal, S. Dahiya, and T. Choudhury, "Energy efficiency in cloud computing data centers: a survey on software technologies," *Cluster Comput.*, vol. 26, no. 3, pp. 1845–1875, 2022, doi: 10.1007/s10586-022-03713-0.
- [2] A. Schuler and G. Kotsis, "A systematic review on techniques and approaches to estimate mobile software energy consumption," *Sustain. Comput. Informatics Syst.*, vol. 41, p. 100919, 2024, doi: 10.1016/j.suscom.2023.100919.
- [3] K. Dutta and D. Vandermeer, "A Case for Sustainability and Environment Friendliness in Software Development and Architecture Decisions by Taking Energy-Efficient Design Decisions," 2023, doi: 10.48550/arXiv.2311.01680.

- [4] C. Pang, A. Hindle, B. Adams, and A. E. Hassan, "What Do Programmers Know about Software Energy Consumption?," *IEEE Softw.*, vol. 33, no. 3, pp. 83–89, 2016, doi: 10.1109/MS.2015.83.
- [5] J. Veiga, J. Enes, R. R. Expósito, and J. Touriño, "BDEv 3.0: Energy efficiency and microarchitectural characterization of Big Data processing frameworks," *Futur. Gener. Comput. Syst.*, vol. 86, pp. 565–581, 2018, doi: 10.1016/j.future.2018.04.030.
- [6] J. Mancebo, C. Calero, and F. García, "Does maintainability relate to the energy consumption of software? A case study," *Softw. Qual. J.*, vol. 29, no. 1, pp. 101–127, 2021, doi: 10.1007/s11219-020-09536-9.
- [7] J. Mancebo, F. García, and C. Calero, "A process for analysing the energy efficiency of software," *Inf. Softw. Technol.*, vol. 134, p. 106560, 2021, doi: 10.1016/j.infsof.2021.106560.
- [8] P. Ciancarini *et al.*, "Analysis of Energy Consumption of Software Development Process Entities," *Electronics*, vol. 9, no. 10, 2020, doi: 10.3390/electronics9101678.
- [9] K. Dutta and D. Vandermeer, "Reduce, Reuse, Recycle: Building Greener Sustainable Software," 2023, doi: abs/2311.01678.
- [10] J. E. Smith and R. Nair, *Virtual Machines*, 1st ed. 2005. doi: 10.1016/B978-1-55860-910-5.X5000-9.
- [11] J. E. Smith and R. Nair, "The Architecture of Virtual Machines," *Computer (Long. Beach. Calif.)*, vol. 38, no. 5, pp. 32–38, 2005, doi: 10.1109/MC.2005.173.
- [12] T. Lindholm, F. Yellin, G. Bracha, and A. Buckley, *The Java Virtual Machine Specification, Java SE 8 Edition*, 1st ed. Addison-Wesley Professional, 2014.
- [13] S. S. Muchnick, *Advanced compiler design and implementation*. Morgan Kaufmann Publishers Inc., 1998.
- [14] R. Pereira *et al.*, "Energy efficiency across programming languages: how do energy, time, and memory relate?," in *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering*, 2017, pp. 256–267. doi: 10.1145/3136014.3136031.
- [15] S. Rabinowitz and S. Wagon, "A spigot algorithm for the digits of pi," in *Pi: The Next Generation: A Sourcebook on the Recent History of Pi and Its Computation*, Springer International Publishing, 2016, pp. 207–217. doi: 10.1007/978-3-319-32377-0_13.
- [16] INTEL, "Running Average Power Limit Energy Reporting." 2017. [Online]. Available: <https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/advisory-guidance/running-average-power-limit-energy-reporting.html>